Social Media Networking and Friend Recommendation System

TEJAS RAICHURA

23BEC154

ANSHPREET PUNJABI

23BEC150

A report submitted for project of DATA STRUCTURE AND ALGORITHM
Taught by Proffesor Dr. PARITA OZA

October 25, 2025

Contents

1 Introduction **1**
1.1 Project Objectives 1
1.2 Core Data Structures 1
1.3 Scope and Methodology 2

2 Algorithms and Data Structures **2**
2.1 Core Data Structures 2
2.2 Dynamic Network Resizing Algorithm 2
2.3 User Deletion Algorithm (deletenode) 3
2.4 Friend Recommendation Algorithm (recommendation) 3

3 Time Complexity Analysis **4**

4 Conclusion and Future Work **5**
4.1 Conclusion 5
4.2 Future Work 5

Abstract

This report presents a Social Network and Friend Recommendation System developed in C. The system uses a graph data structure, implemented via an adjacency list, to represent users as nodes and friendships as edges. Dynamic memory allocation (malloc/realloc) is employed to allow the network's capacity to scale efficiently as new users are added. Key features include complete user management (register, delete, view profile) and connection management (add/delete friendships). The core of the project is a "Friends of Friends" algorithm, which analyzes the graph to suggest new connections to a user based on their second-degree relationships. The entire system is operated through a menu-driven command-line interface.

1 Introduction

1.1 Project Objectives

The primary objective of this project is to design, build, and test a C-based simulation of a social media network with a friend recommendation feature. The specific goals are to:

1. Implement a Graph Data Structure: Utilize an adjacency list to represent the social graph, as it is highly efficient for sparse networks.
2. Ensure Scalability: Employ dynamic memory allocation (malloc, realloc) to allow the network to grow beyond its initial capacity.
3. Develop Core User Functionality: Create functions to register new users, delete existing users, and view detailed user profiles.
4. Manage Network Connections: Implement logic to create (add) and sever (delete) bidirectional friendships between users.
5. Build a Recommendation Engine: Implement a "Friends of Friends" (FoF) algorithm to analyze the graph and suggest potential new friends to a user.

1.2 Core Data Structures

To achieve its objectives, the system is built upon a few key data structures in C:

- UserProfile (struct): A structure that holds all information for a single user, such as their userID, name, age, email, and city.
- Node (struct): The building block for the adjacency list. Each Node contains a user's ID (vertex) and a pointer (next) to the next friend in the list.
- Node **adjlist (Adjacency List): The primary data structure for the graph. This is a dynamic array of pointers, where each index i corresponds to a user. adjlist[i] points to the head of a linked list of Node structs, with each node in that list representing a friend of user i.
- Parallel Dynamic Arrays: Two other dynamic arrays are maintained in parallel with the adjacency list:
  - user *profiles: Stores the UserProfile structs. profiles[i] contains the data for user i.
  - bool *registered: A boolean array used as a status map. registered[i] is true if the user exists and false if they have been deleted.

1.3 Scope and Methodology

This project is implemented as a single-file, menu-driven application in C. The user interacts with the system via a Command-Line Interface (CLI). The scope is focused on the backend logic and data structure management rather than a graphical user interface or database persistence.

The system's methodology is as follows:

- User Profiles and the Graph Structure are stored in the dynamic arrays and adjacency list described above.
- The registered array acts as a lookup table, allowing for efficient $O(1)$ status checks and enabling user deletion without needing to shift all elements in the arrays.
- All user-facing operations (e.g., "add connection," "find recommendations") are handled by name-based "wrapper" functions (like addConnectionByName), which first find the user's internal ID and then call the core logic functions.

2 Algorithms and Data Structures

To build a scalable and efficient social network, this project relies on a specific set of data structures and algorithms. The core of the system is a graph, which is managed using dynamic arrays and adjacency lists.

2.1 Core Data Structures

1. **Graph Representation (Adjacency List):** The social network is modeled as an undirected graph. The primary data structure is an adjacency list, implemented as Node **adjlist. This is a dynamic array of pointers. Each index i in this array (adjlist[i]) corresponds to the user with userID i and stores a pointer to the head of a linked list. This linked list contains Node structures, with each node representing a "friend" (an edge) by storing their vertex (user ID).
2. **Parallel Data Arrays:** To store user information and manage the graph's state, two other dynamic arrays are used in parallel with adjlist:
   - **user *profiles:** An array where profiles[i] stores the UserProfile struct (name, age, city, etc.) for the user with userID i. This allows for $O(1)$ (constant time) lookup of a user's profile once their ID is known.
   - **bool *registered:** A boolean "lookup" array. registered[i] == true signifies that user i is an active member of the network. This is crucial for the deletion algorithm.

2.2 Dynamic Network Resizing Algorithm

To avoid a fixed user limit, the system automatically scales its capacity.

Trigger: This algorithm is called by registerUserByName() when the number of users (user_count) equals the current capacity.

Process:
1. A new, larger capacity is calculated (e.g., new_capacity = capacity * 2).
2. The realloc function is used to resize all three dynamic arrays (registered, profiles, and adjlist) to this new capacity. realloc efficiently preserves all existing user data.

3. The newly allocated portion of the arrays is initialized (e.g., new registered slots are set to false).
4. The global capacity variable is updated.

Benefit: This gives the system amortized $O(1)$ insertion time for new users, making it highly scalable.

2.3 User Deletion Algorithm (deletenode)

Deleting a user (a node) from the graph is a multi-step process that must preserve the graph's integrity.

1. **Soft Deletion:** The user n is marked as inactive by setting registered[n] = false. This immediately and effectively removes them from all future operations (like findUserIdByName or recommendation) without needing to shift elements in the arrays.
2. **Remove Incoming Edges:** The algorithm iterates through every other registered user i in the network (from 0 to user_count). For each user i, it calls removeEdge(i, n), which searches adjlist[i] for a node pointing to n and removes it. This is essential to prevent other users from having "dangling" connections to a deleted user.
3. **Free Outgoing Edges:** The algorithm frees the deleted user's own adjacency list (adjlist[n]). It traverses the entire linked list of n's friends and frees each Node, preventing memory leaks.

2.4 Friend Recommendation Algorithm (recommendation)

This is the system's core feature, which implements a "Friends of Friends" (FoF) algorithm. The goal is to find all users who are two "steps" away from the user (Level 2 connections) but are not already one "step" away (Level 1 connections).

The algorithm proceeds in the following steps for a given user_ID:

1. **Initialization:** Two temporary boolean arrays, isfriend and recommended, are allocated with size capacity (using calloc to initialize all values to false).
   - isfriend will be used to mark the user and all their direct friends (L1).
   - recommended will be used to track who has already been suggested to prevent duplicates.
2. **Mark L1 Friends:** The algorithm first traverses the user's own adjacency list (adjlist[user_ID]). For every direct friend f it finds:
   - It marks them in the tracking array: isfriend[f] = true.
   - It also marks the original user: isfriend[user_ID] = true.
3. **Find L2 Friends:** The algorithm then begins its main traversal. It iterates through the user's L1 friends again. For each L1 friend (L1_friend), it traverses their adjacency list (adjlist[L1_friend->vertex]). Each person in this second-level list is a potential recommendation (L2_friend).
4. **Filtering Logic:** For each L2_friend, two checks are performed:
   - if (!isfriend[L2_friend]): This is the most important check. It verifies that the L2_friend is not the original user and not already a direct friend.

- if (!recommended[L2_friend]): This check ensures this L2_friend hasn't already been suggested (which can happen if they are a mutual friend of two or more L1 friends).

5. **Output:** If both checks pass, the L2_friend's name is printed as a recommendation, and they are marked as recommended[L2_friend] = true.
6. **Cleanup:** Finally, the temporary isfriend and recommended arrays are freed to prevent memory leaks.

3 Time Complexity Analysis

The efficiency of the social network system can be analyzed using Big O notation. In this analysis, the following variables are used:

- N: The total number of registered users (user_count).
- C: The current capacity of the dynamic arrays.
- E: The total number of connections (edges) in the graph.
- d: The maximum degree of a node (i.e., the maximum number of friends any single user has).

The time complexity for each key operation is as follows:

Find User By Name (findUserIdByName): $O(N)$ **Justification:** This function performs a linear search through the profiles array from 0 to user_count. In the worst case, it must check all N users.

Register User (registerUserByName): $O(N)$ **(Amortized) Justification:** The dominant cost is the $O(N)$ call to findUserIdByName to check for duplicate names. The insertion of the new user is $O(1)$ amortized. While the resizeNetwork function takes $O(C)$ time, it only occurs when $N = C$, and its cost is "spread out" over many $O(1)$ insertions.

View Profile (viewProfileByName): $O(N)$ **Justification:** The operation is completely dominated by the $O(N)$ cost of findUserIdByName to locate the user. Accessing the profile data afterward is $O(1)$.

Add Connection (addConnectionByName): $O(N + d)$ **Justification:** This requires two $O(N)$ calls to findUserIdByName. It then scans the adjacency list of the first user to check for duplicates, which takes $O(d)$ time. The subsequent addEdge operations are $O(1)$ (insertion at the head of a linked list). Total: $O(N) + O(N) + O(d) \approx O(N + d)$.

Delete Connection (deleteConnectionByName): $O(N + d)$ **Justification:** Similar to adding, this requires two $O(N)$ lookups. The removeEdge function must then scan the adjacency lists of both users, each taking $O(d)$ time in the worst case. Total: $O(N)+O(N)+O(d) \approx O(N + d)$.

Display Network (display): $O(N + E)$ **Justification:** This function iterates through all N users in the main array. It also traverses the entire adjacency list for each user. The sum of the lengths of all adjacency lists is $2E$ (since each edge is stored twice). This is the standard, most efficient time complexity for a full traversal of a graph.

Delete User (deleteNodeByName): $O(N + E)$ **Justification:** This is the most computationally expensive operation. After an $O(N)$ lookup, the deletenode function must iterate through all N users. For each user i, it calls removeEdge(i, n), which may scan adjlist[i].

The total work for this loop is $O(N + E)$ (the $O(N)$ for the loop itself plus $O(E)$ for the total work of all removeEdge scans). Finally, it frees the deleted user's own list, which is $O(d)$. The dominant term is $O(N + E)$.

Friend Recommendation (recommendationByName): $O(C + d^2)$ **Justification:**

1. findUserIdByName takes $O(N)$.
2. Allocating the isfriend and recommended arrays takes $O(C)$ time.
3. Marking all L1 (direct) friends takes $O(d)$ time.
4. The main nested loop iterates d times (for each L1 friend). Inside that loop, it iterates d times (for each L2 friend). This results in a $O(d \times d)$ or $O(d^2)$ complexity.

The total is $O(N) + O(C) + O(d) + O(d^2)$. Since $C \geq N$, this simplifies to $O(C + d^2)$. In a dense graph where d can approach N, this complexity can become $O(N^2)$.

4 Conclusion and Future Work

4.1 Conclusion

This project successfully demonstrated the design and implementation of a scalable Social Network and Friend Recommendation System using the C programming language. By leveraging fundamental data structures, the system effectively models the core functionalities of a modern social media platform.

The use of an adjacency list proved to be an efficient choice for representing the graph of user connections, while dynamic memory allocation (using malloc and realloc) was critical in ensuring the system's scalability, allowing it to handle a growing user base without a predefined limit.

All primary objectives were met:

- Users can be dynamically registered and deleted. Friendship connections can be created and removed.
- The "Friends of Friends" (FoF) recommendation algorithm was successfully implemented, providing a practical method for network growth and user engagement.

The menu-driven command-line interface provides a robust platform for testing and interacting with all backend logic, proving the viability of C for managing complex, linked data structures in a memory-efficient manner.

4.2 Future Work

While this project provides a strong foundation, several enhancements could be made in the future:

- **Data Persistence:** Implement functionality to save the current network state (all user profiles and connections) to a file and load it back into memory upon startup.
- **Advanced Recommendation Algorithms:** The FoF algorithm could be improved. A new algorithm could rank recommendations based on the number of mutual friends or by suggesting users who live in the same city.
- **Search and Traversal:** Implement graph traversal algorithms like Breadth-First Search

(BFS) or Depth-First Search (DFS) to find the shortest path (degrees of separation) between two users.

- **Error Handling:** Improve the robustness of user input, for example, by handling non-integer inputs in the menu or validating email formats.